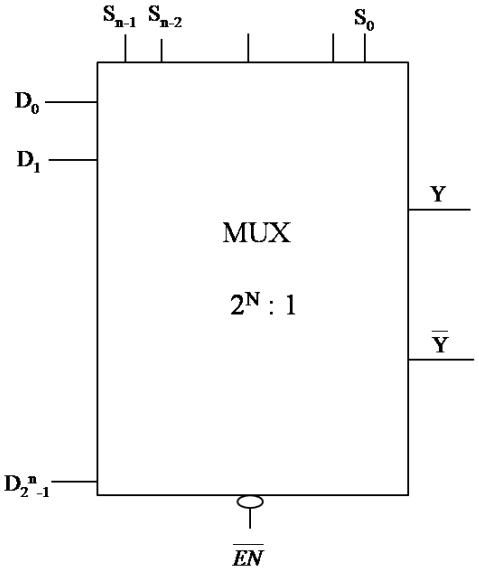
Proiectare Logică

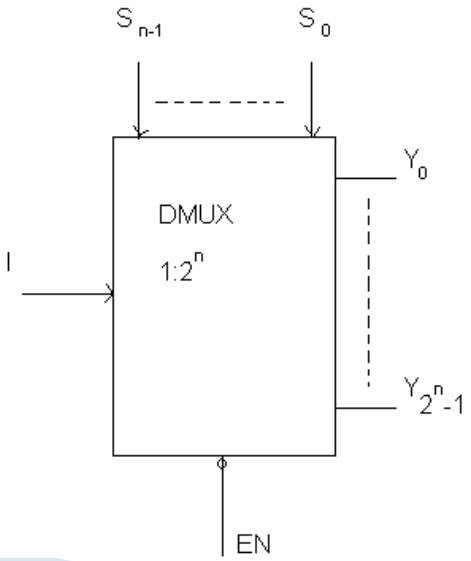
I CA & CB: Mariana Mocanu
mariana.mocanu@cs.pub.ro
I CC & CD: Costin Chiru costin.chiru@cs.pub.ro

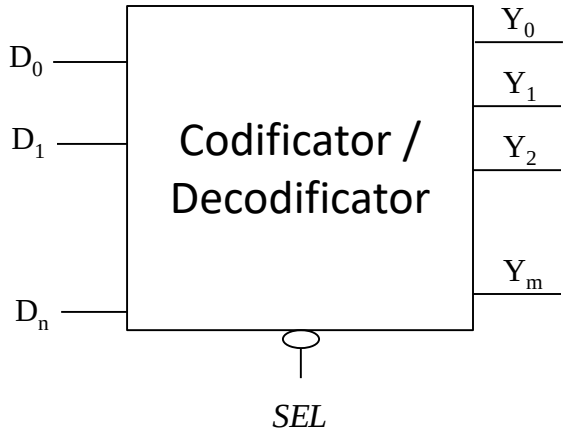
Curs 9: Codificatoare/Decodificatoare. Numărătoare. Registre.

Codificatoare/Decodificatoare

- Circuite combinaționale integrate pe scară medie
- Similare cu MUX/ DMUX
- Majoritatea lucrează în logică negativă (active pe 0)
  - În cazul unui decodificator semnalele de intrare aplicate căilor de adresare determină tranziţia uneia din ieşiri în 0 logic.





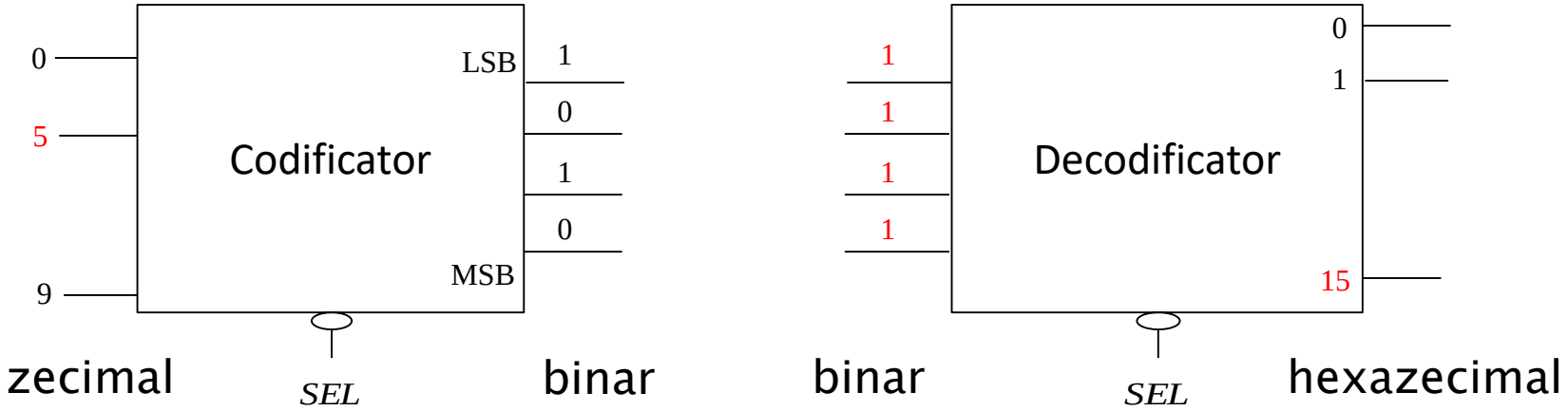


Codificatoare / Decodificatoare

- *Codificatoare*: Vrem să obținem la ieșire codul binar al cifrei introduse la intrare
- Ex: la tastaturi (telefon)
- *Decodificatoare*: la intrare avem un anumit cod și la ieșire generează semnal pe o singură ieșire, cea corespunzătoare codului de la intrare (de la o reprezentare compactă a informației se obține una și doar una dintre porți activă)
- Ex: instalația de luminițe pentru brad când vrem să aprindem un anumit bec (serie de becuri) și nu alta / toate.

Codificatoare / Decodificatoare

- E clar principiul de funcționare și diferența dintre ele (și MUX/DMUX)?



- Există o legătură între numărul de intrări și numărul de ieșiri:
  - Mux: 2^{sel} = intrări, Codificatoare: $2^{intrări}$ = ieșiri
  - La codificatoare nu mai există intrări de selecție ci numai un semnal care îl lasă să funcționeze sau nu

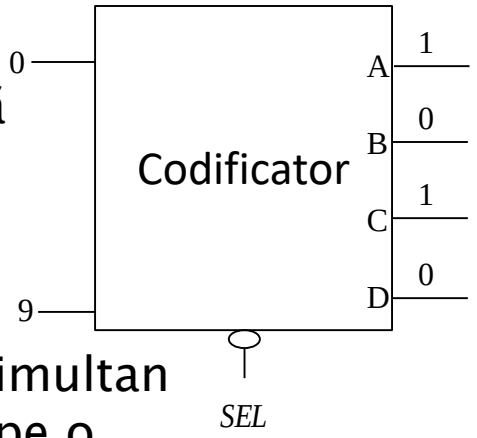
Decodificatoare

- Trecem la exerciții: insistăm pe decodificator pentru că e cel care e utilizat la ieșirile circuitelor secvențiale.
- În activitățile curente folosim baza 10 → avem nevoie de codificator zecimal-binar
  - Considerăm logica pozitivă (în cataloage sunt în logică negativă)

i_9	i_8	i_7	...	i_1	i_0	D	C	B	A
0	0	0	...	0	0	–	–	–	–
0	0	0	...	0	1	0	0	0	0
0	0	0	...	1	0	0	0	0	1
0	0	0	...	1	1	*	*	*	*

Câte linii în tabel?
Câte ne interesează
→invalidă

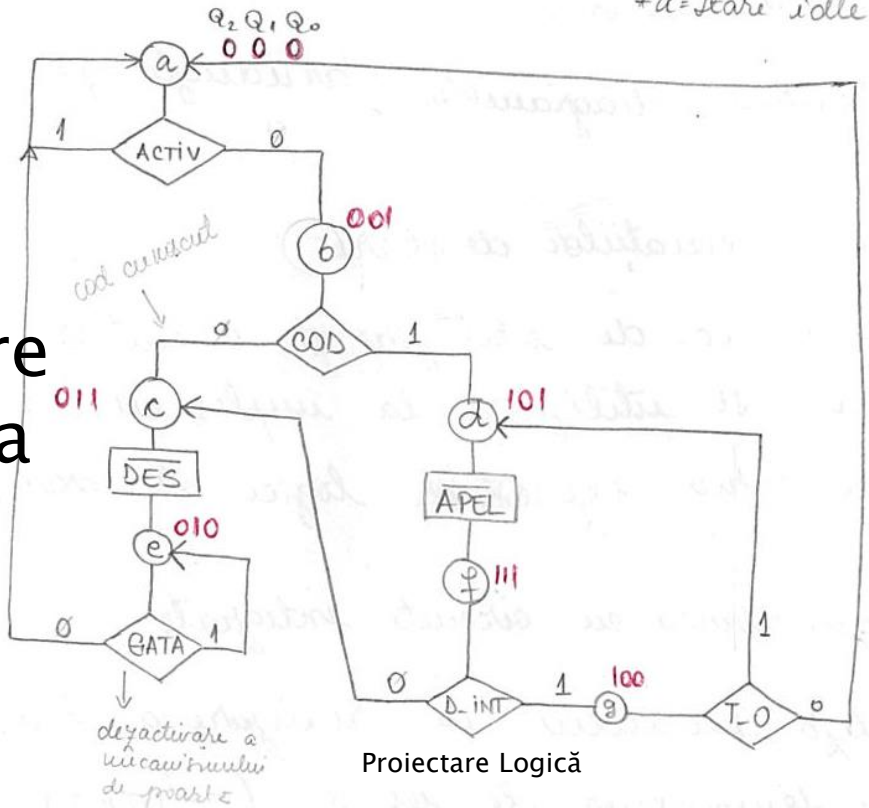
2^{10}
10



nu apăs 2 taste simultan (primesc semnal pe o singură intrare)

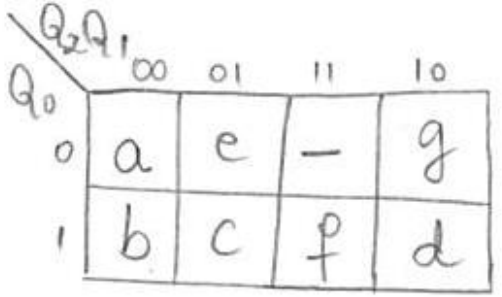
Decodificatoare

- Există binar – hexa, binar – zecimal (ultimele 6 ieșiri nu mai apar), binar -12
- De ce avem nevoie de decodificatoare?
- Revenim la problema cu interfonul
- Când se genera !DES / !APEL?
- În c / d
- Semnalele de ieșire se pot implementa cu porți dar și cu decodificator



Decodificatoare

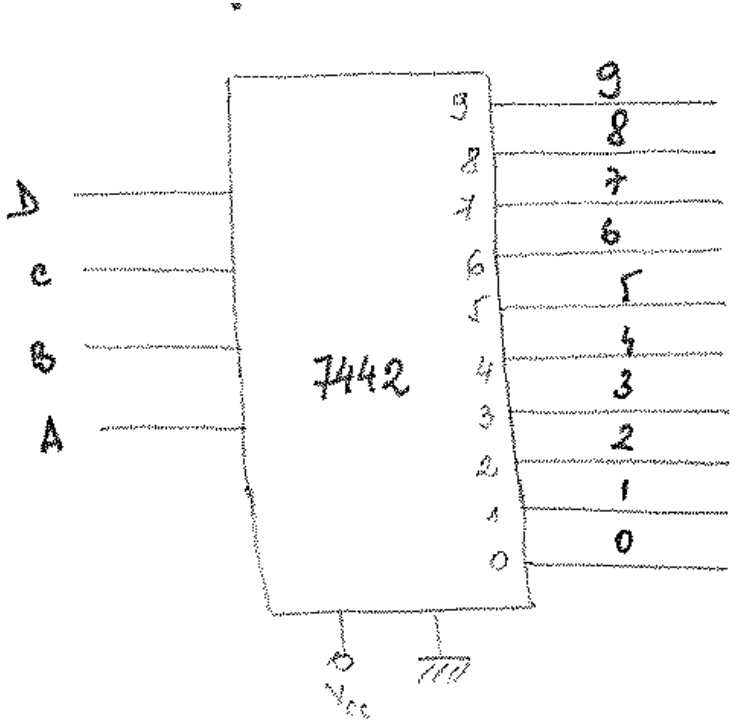
- Cum se face implementarea ieșirilor folosind decodificator?
- Dacă se pun variabilele de stare pe intrare, la ieșire se activează una și numai una din ieșiri
- Pentru a obține !DES la ieșirea decodificatorului, trebuie să avem la intrare 011, iar pentru !APEL vom avea 101
- Observație: dacă avem mai multe ieșiri, este necesar 1 SINGUR decodificator pentru a le implementa pe toate!



Decodificatorul 7442

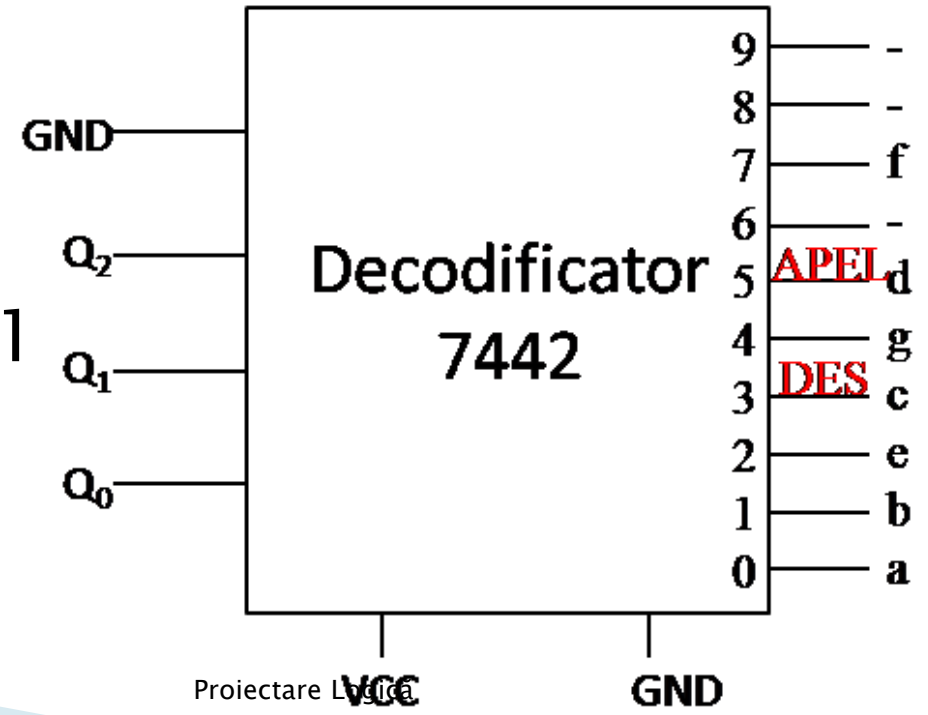
- Folosit pentru decodificarea spațiului stărilor în circuitele secvențiale și pentru generarea ieșirilor la circuite de tip Moore
- Nu are nevoie de EN, ci doar de VCC și GND
- Activ pe 0

D	C	B	A	0	1	2	3	4	5	6	7	8	9
0	0	0	0	0	1	1	1	1	1	1	1	1	1
0	0	0	1	1	0	1	1	1	1	1	1	1	1
0	0	1	0	1	1	0	1	1	1	1	1	1	1
0	0	1	1	1	1	1	0	1	1	1	1	1	1
0	1	0	0	1	1	1	1	0	1	1	1	1	1
.	.	.	.	.	.	.	.	.	.	.	.	.	.



Decodificatorul 7442

- Ce se întâmplă dacă pe intrări pun D – GND, C – Q_2, B – Q_1, A – Q_0?
- În enunțul problemei am cerut ca ieșirile să fie active pe 0 → exact cum sunt generate de 7442
- → ieșirile stau în 1 atâta timp cât decodificatorul nu primește pe intrare 011 (când DES devine 0) sau 101 (când APEL devine 0)

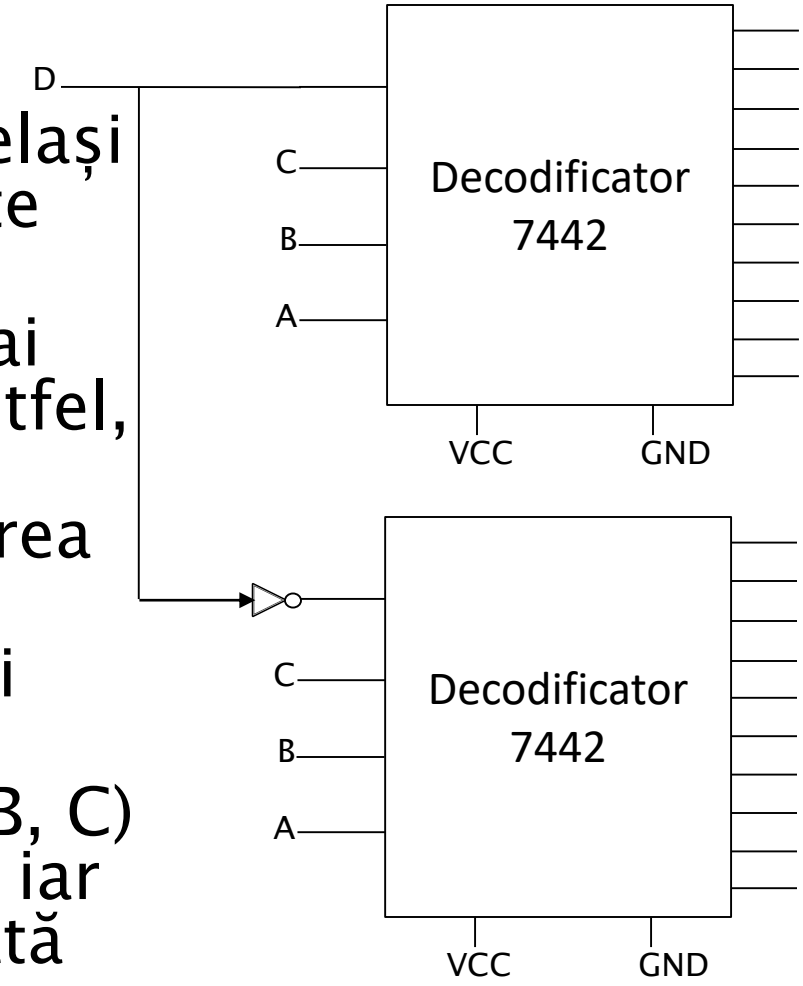


Utilitatea decodificatoarelor

1. Există multe situații în care nu am acces la stările circuitelor, dar sunt și situații când am acces la ele → în acest caz, în partea de proiectare, circuitele se testează folosind testere ce conțin astfel de decodificatoare (ce au și ieșiri luminoase) a.î. dacă generez manual impulsuri de ceas să se poată verifica trecerea aparatului dintr-o stare în alta
2. Generarea semnalelor de ieșire (decodifică spațiul stărilor) – foarte simplă, ca în exemplu

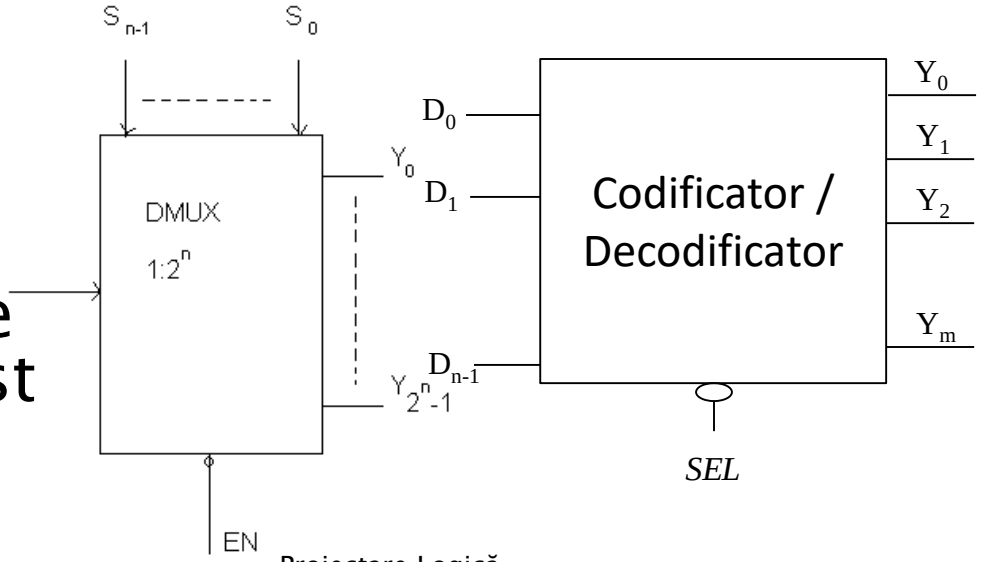
Decodificatoare

- Ce se întâmplă dacă am același semnal generat în mai multe stări?
- Pun SAU între ele, dar numai dacă e în logică pozitivă. Altfel, pun ȘI-NU în loc de SAU (e important să știm dacă ieșirea este pe 0 sau 1)
- Cum putem să scoatem mai multe ieșiri?
- Prin legarea în cascadă (A, B, C) se leagă la ambele aparate, iar D intră o dată direct și o dată negat



Decodificatoare / demultiplexoare

- Cu excepția lui 7442 care nu are intrări de selecție, decodificatoarele au o intrare de selecție (dacă e activat, atunci în funcție de intrare se activează o ieșire).
- În cataloage nu veți găsi separat decodificatoarele și DMUX, ci le veți găsi în aceeași categorie decodificatoare / DMUX
- Transformarea decodificatorului în DMUX se face prin aplicarea pe intrări a variabilelor de selecție și folosirea SEL pe post de semnal de date.



Decodificatoare / demultiplexoare

- Se preferă utilizarea DMUX în cazul în care nu sunt prea multe funcţii, deoarece chiar dacă folosirea DMUX necesită mai puţine porţi suplimentare, pentru fiecare funcţie este necesar un DMUX.
- Utilizarea unui decodificator se preferă în cazul existenţei multor funcţii deoarece ieşirile unui singur decodificator pot fi folosite pentru generarea unui număr mare de funcţii.
- Cum obținem semnalele de ieșire folosind decodificator?
  - Identificăm semnalele care ies din ASM
  - Din organigramă identificăm în ce stări se generează fiecare semnal de ieșire
  - Semnalele de ieșire vor fi pe pozițiile corespunzătoare stărilor identificate

Semnale de ieșire cu porți

- Se realizează făcând diagramele Karnaugh

Q_0 \ Q_2Q_1	00	01	11	10
0	0	0	*	0
1	0	1	0	0

$DES = !Q_2Q_1Q_0$

Q_0 \ Q_2Q_1	00	01	11	10
0	0	0	*	0
1	0	0	0	1

$APEL = Q_2!Q_1Q_0$

- Cum ne interesează să fie active pe 0, trebuie să negăm rezultatele sau să facem diagramele negate

Q_0 \ Q_2Q_1	00	01	11	10
0	1	1	*	1
1	1	0	1	1

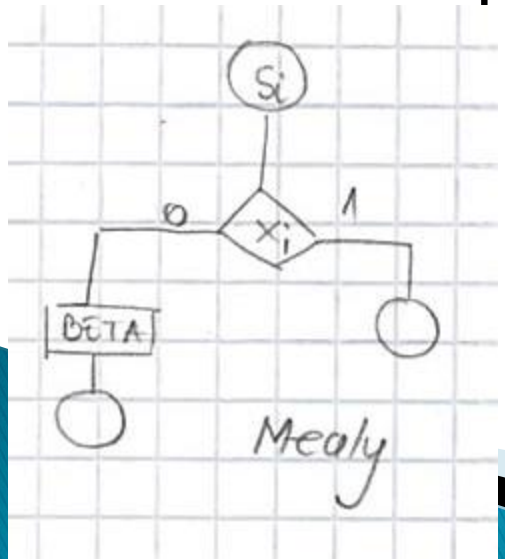
$!DES = Q_2 + !Q_1 + !Q_0$

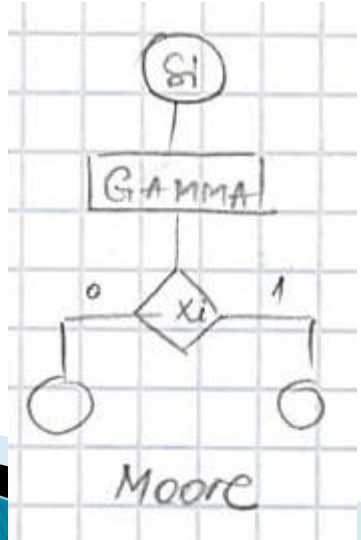
Q_0 \ Q_2Q_1	00	01	11	10
0	1	1	*	1
1	1	1	1	0

$!APEL = !Q_2 + Q_1 + !Q_0$

Semnale de ieșire cu porți

- Dacă ieșirea se generează în mai multe stări, punem 1 în fiecare stare
- Diferență: Ieșirea se generează mereu la Moore, în timp ce la Mealy se generează doar când $x_i = 0$
- În viață lucrurile sunt mai complicate, așa că circuitele pot conține atât zone de tip Mealy cât și zone de tip Moore.





Q_0 \ Q_2Q_1	00	01	11	10
0	$!x_i$			
1				

Q_0 \ Q_2Q_1	00	01	11	10
0	1			
1				

Numărătoare

- Numărătoarele sunt o clasă foarte importantă de circuite secvenţiale de ordin 2 având la bază circuitele basculante bistabile.
- Numărătoarele se găsesc sub formă de circuite integrate de tip MSI sau pot fi sintetizate în laborator (se pot realiza) din CBB-uri.
- Realizează o secvență de numărare.
- Numărătoarele prezintă un număr de ieşiri care prin valoarea lor logică indică numărul de impulsuri recepţionate pe intrări, din momentul unei stări iniţiale cunoscute.
- *Capacitatea numărătorului* se defineşte în raport cu numărul de ieşiri: un numărător cu n ieşiri are capacitatea maximă de 2^n.
- Cele mai utilizate numărătoare sunt cele binare, în care la ieşire avem codul binar al stării numărătorului.

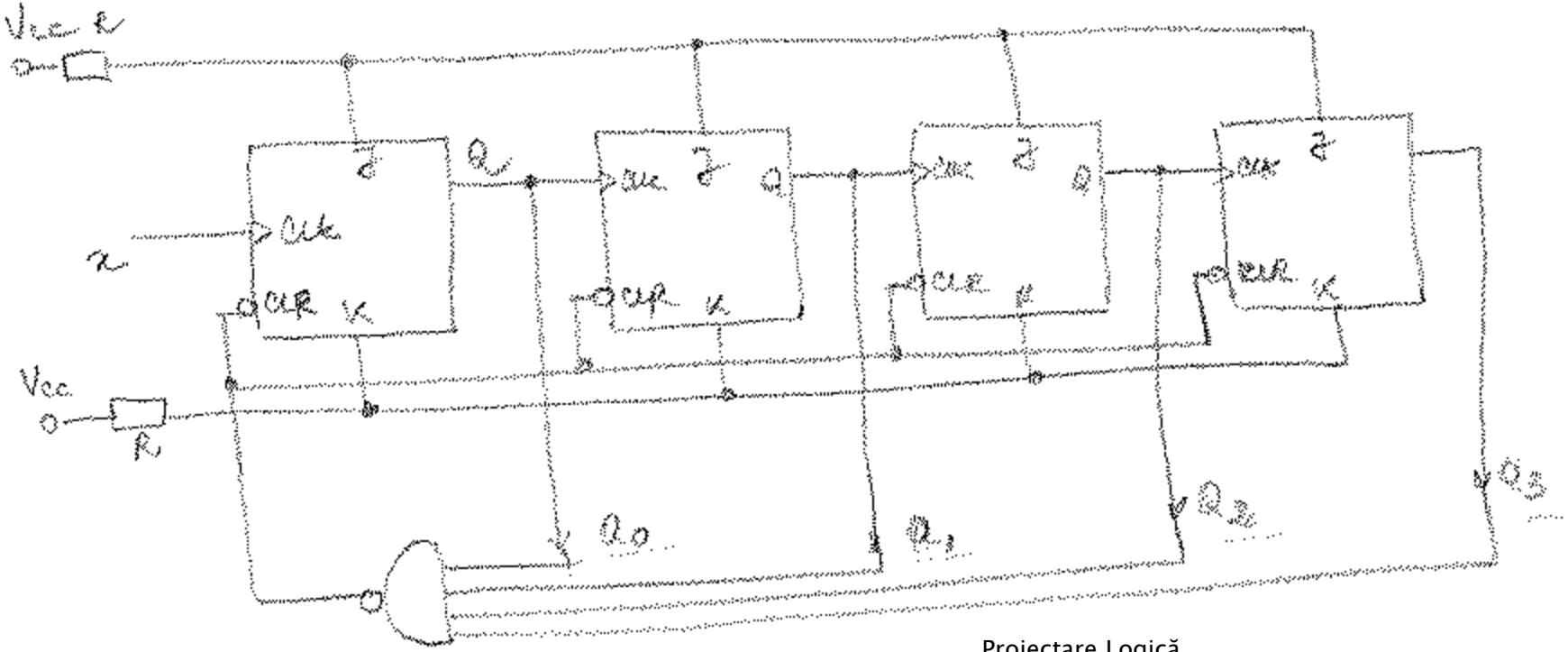
Clasificarea numărătoarelor

1. După secvenţa de numărare: hexazecimale, zecimale, 12
2. După modul de aplicare al semnalului de tact:
   - numărătoare asincrone (impulsul de tact nu se aplică în acelaşi moment la toate automatele elementare) și
   - numărătoare sicrone (impulsul de tact se aplică simultan tuturor automatelor elementare care compun numărătorul)

Numărătoare

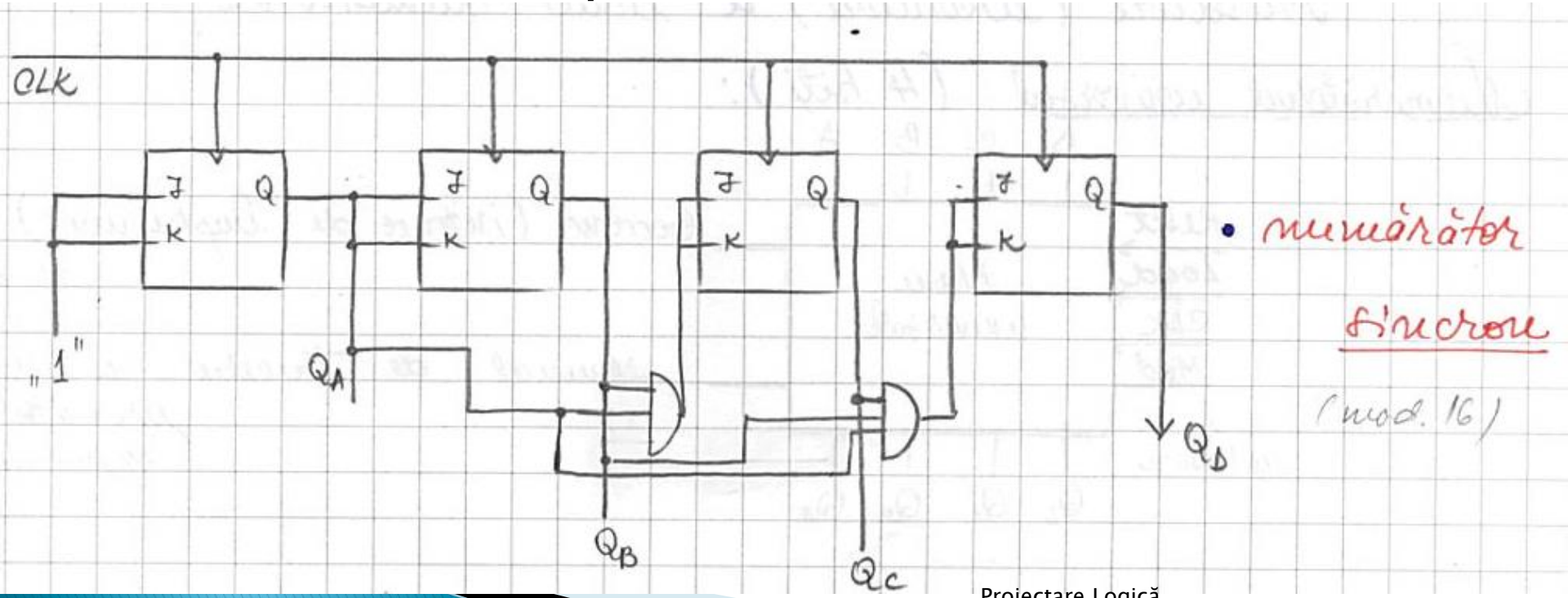
- E sincron sau nu?
- Intrarea poate să fie de forma:
- Ieșirea este asincronă pentru că momentul la care vin semnalele de intrare e aleator





Numărătoare

- Ca să devină sincron, aparatul trebuie modificat ca mai jos:
- Cel asincron e mai ușor de realizat, dar dacă am > celule, e posibil să avem întârzieri

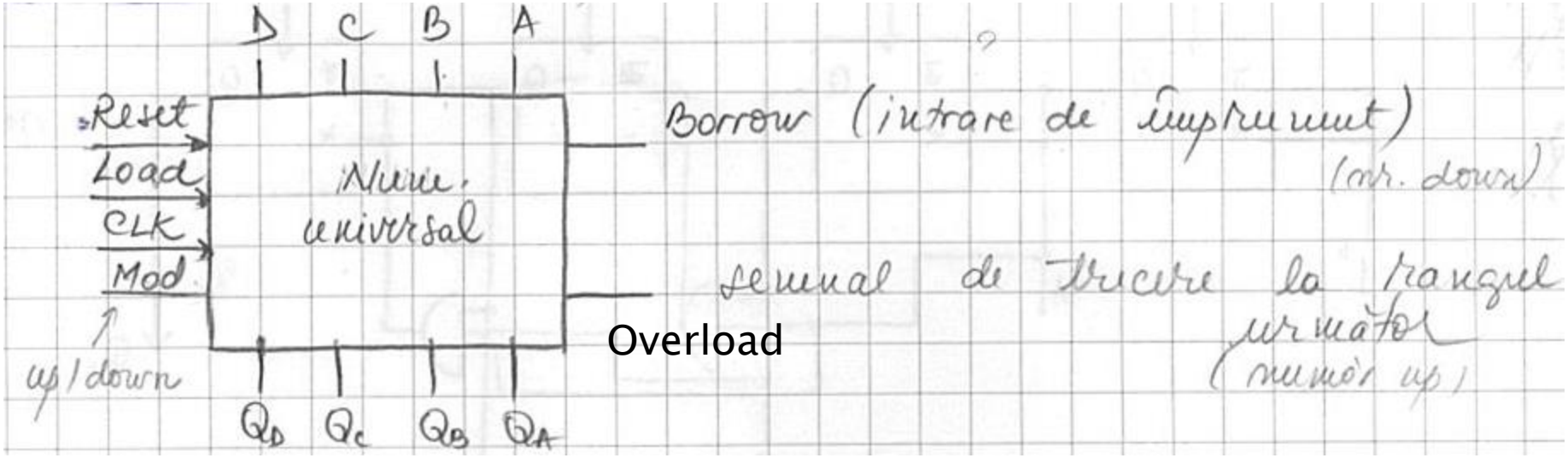


Clasificarea numărătoarelor

3. După modul de numărare:
   - direct (1 → 16),
   - invers (16 → 1), foarte ușor de realizat pentru că CBB-urile au și Q și !Q și pentru numărare inversă luăm !Q în loc de Q
   - în ambele sensuri
4. După ciclicitate: ciclice sau nu (trebuie resetate în mod explicit pentru a reveni la starea inițială)

Numărătorul universal (4 biți)

- Exemplu de numărător – există constructiv (pe capsulă)



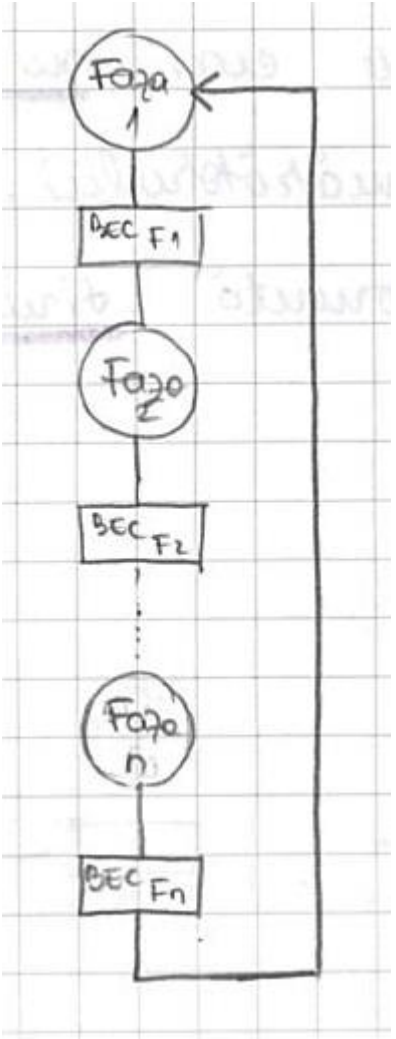
- Overload permite concatenarea mai multor numărătoare universale ca să nu mai numere pe 4 biți, ci pe mai mulți

Funcţii ale numărătorului universal

- Funcţia de numărare, în sens direct sau invers
- Funcţia de încărcare a unei valori iniţiale (presetare) – dacă vrem să pornim de la o anumită valoare (în loc de 0), aceasta se face prin activarea lui LOAD ce încarcă în numărător în mod paralel (toate odată) valorile D, C, B, A
- Funcţia de ştergere a conţinutului (resetare sau aducere la zero)

Numărătoare în loc de CBB-uri

- Pentru organigrame foarte simple
- Ex: semaforul
- Secvență de numărare modulo n fără salturi, foarte simplă
- Cum fac implementarea?
- Iau numărătorul, bag ieșirile în decodificator și astfel se aprinde pe rând doar un anumit set de becuri.



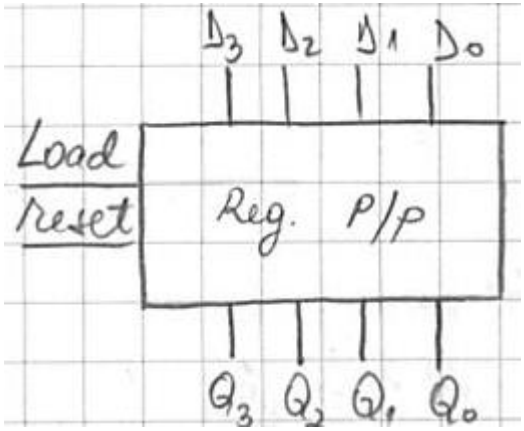
Registre

- Ați auzit de registre? Unde? Ce sunt?
- Ați auzit că unitatea centrală a unui calculator conține obligatoriu niște regiștrii și UAL?
- Reprezintă o clasă de circuite secvențiale realizate pe baza CBB-urilor cu scopul de a memora și pune la dispoziție informația formată din semnale elementare.
- Numărul semnalelor elementare este egal cu cel al circuitelor basculante bistabile care compun registrul
- Capacitatea registrului = numărul de biți care pot fi stocați sau memorați în registru
- Se aseamănă cu numărătoarele, dar salvează informația pe > ranguri binare, nu doar pe 1

Clasificare registre

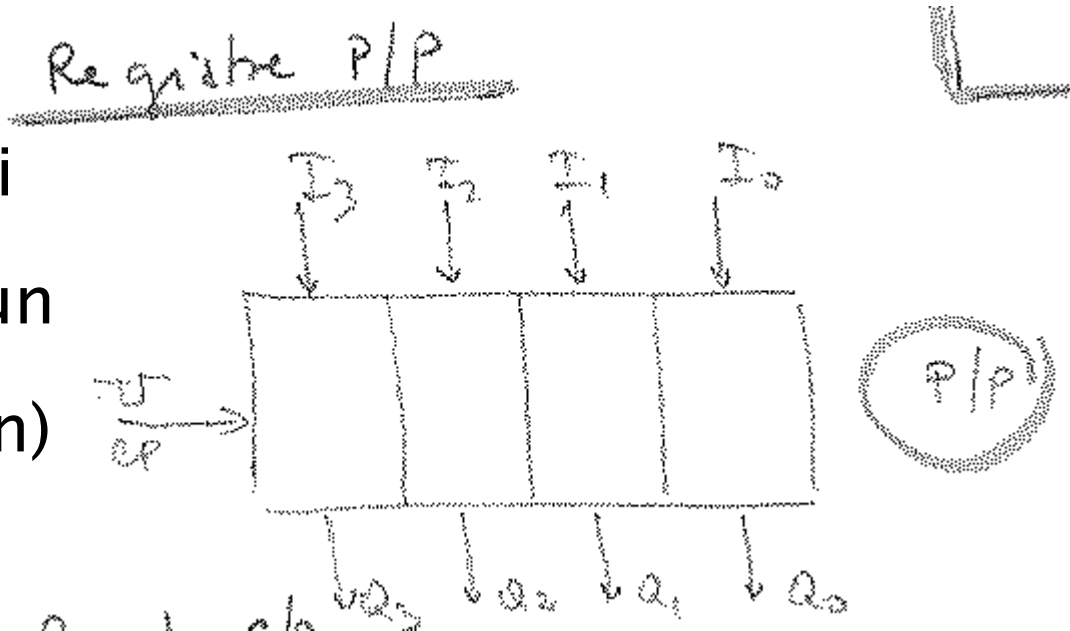
- În funcție de modul de realizare al intrărilor și ieșirilor (serie / paralel)

Intrare	Ieșire
P	P
P	S
S	P
S	S



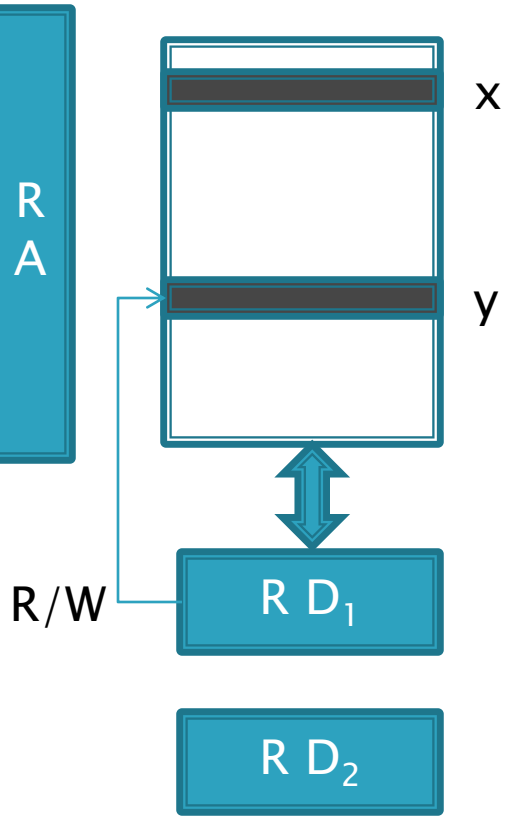
Registre P/P

- La Load încarcă intrările în registru și le pune la dispoziția ieșirilor până la un nou load
- Seamănă cu funcția de presetare de la numărătorul universal
- Registrele sunt foarte utile în anumite zone ale calculatorului, dar și în interacțiunea dintre calculator și dispozitive periferice
- Ex: imprimanta
  - Acum: USB
  - În trecut:conectori cu mulți pini → comunicare P-P (un cuvânt se putea transmite simultan)



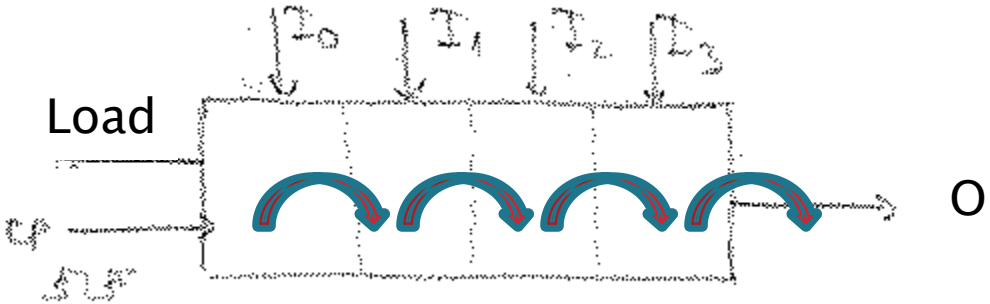
Registre P/P

- Cum e organizată memoria?
- Pe adrese, la anumite locații avem diverse cuvinte
- Dacă vrem să facem operații cu 2 cuvinte din memorie, unul trebuie să fie salvat undeva → unde? în registrul de date
- De asemenea, există registru de adrese
- Avem nevoie de registre P/P pentru a prelua informația, a o stoca pentru o anumită perioadă de timp pentru ca ulterior să o pună la dispoziție.

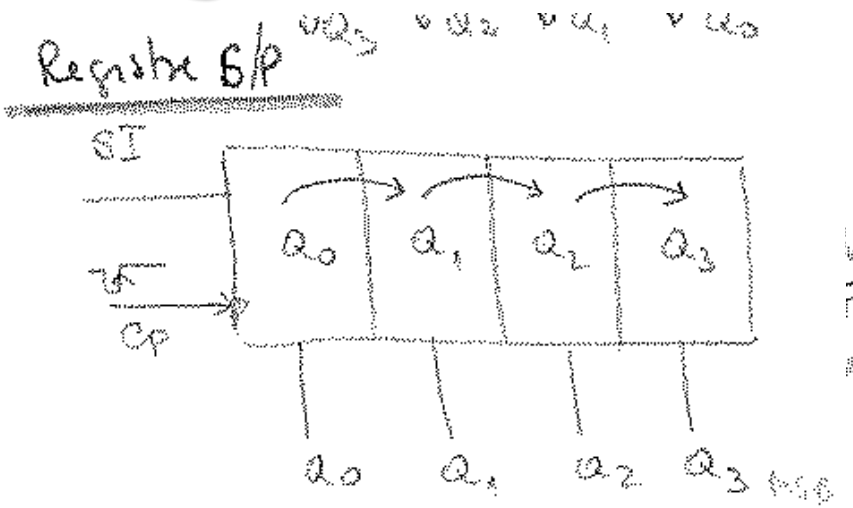


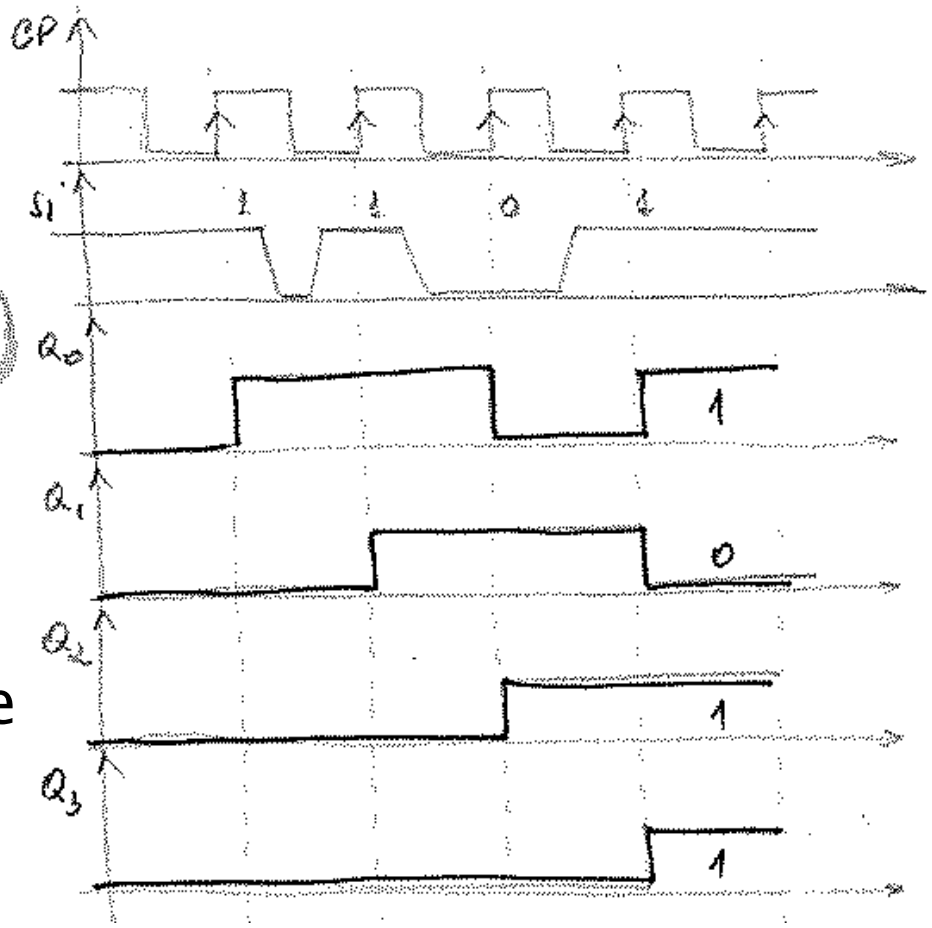
Registre P/S

- Serializare – serializează semnalul de intrare
- date de la intrare încărcate paralel
- Prelucrarea se face bit cu bit
- În calculator toate operațiile se fac în mod paralel – serializarea se face pentru transmiterea datelor (ex: imprimantă, fibră optică) → la celălalt capăt semnalul trebuie deserializat cu ajutorul registrelor S/P



Registre S/P



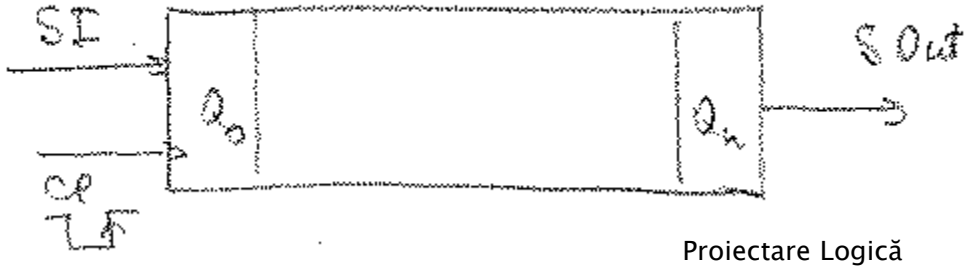


- Rol:deserializarea datelor recepționate
- Alt rol: sincronizarea dintre aparate ce funcționează la frecvențe diferite (ex: procesor și dispozitive auxiliare)
- Buffer – transportă informația între aparate cu frecvențe diferite

Ex: memorarea numărului 1101

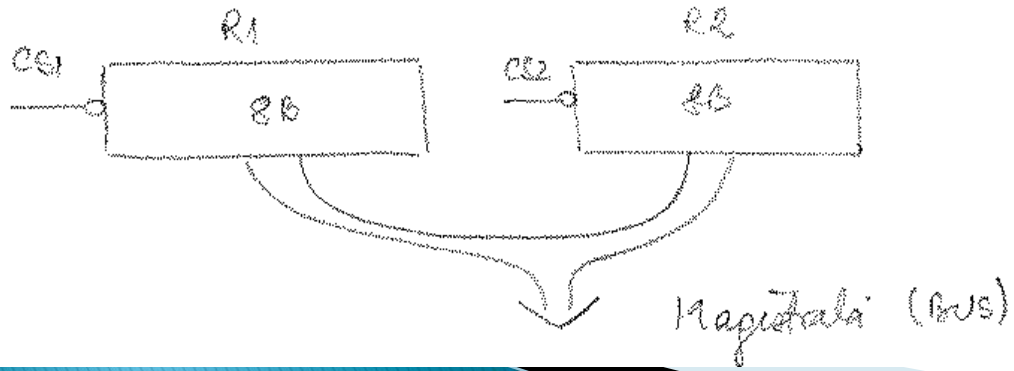
Registre S/S

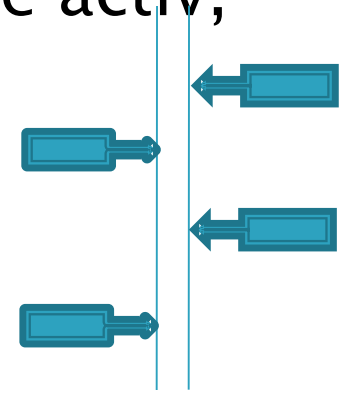
- Cozile de la programare (FIFO)
- Aceste registre se pot construi pe baza CBB-urilor
- Semnalul este transmis sincron în funcție de semnalul de ceas
- Accesul la informație este posibil doar după n tacturi de ceas (n = capacitatea registrului)
- Există și registre care funcționează după principiul stivei (LIFO)



Registre three-state (TS)

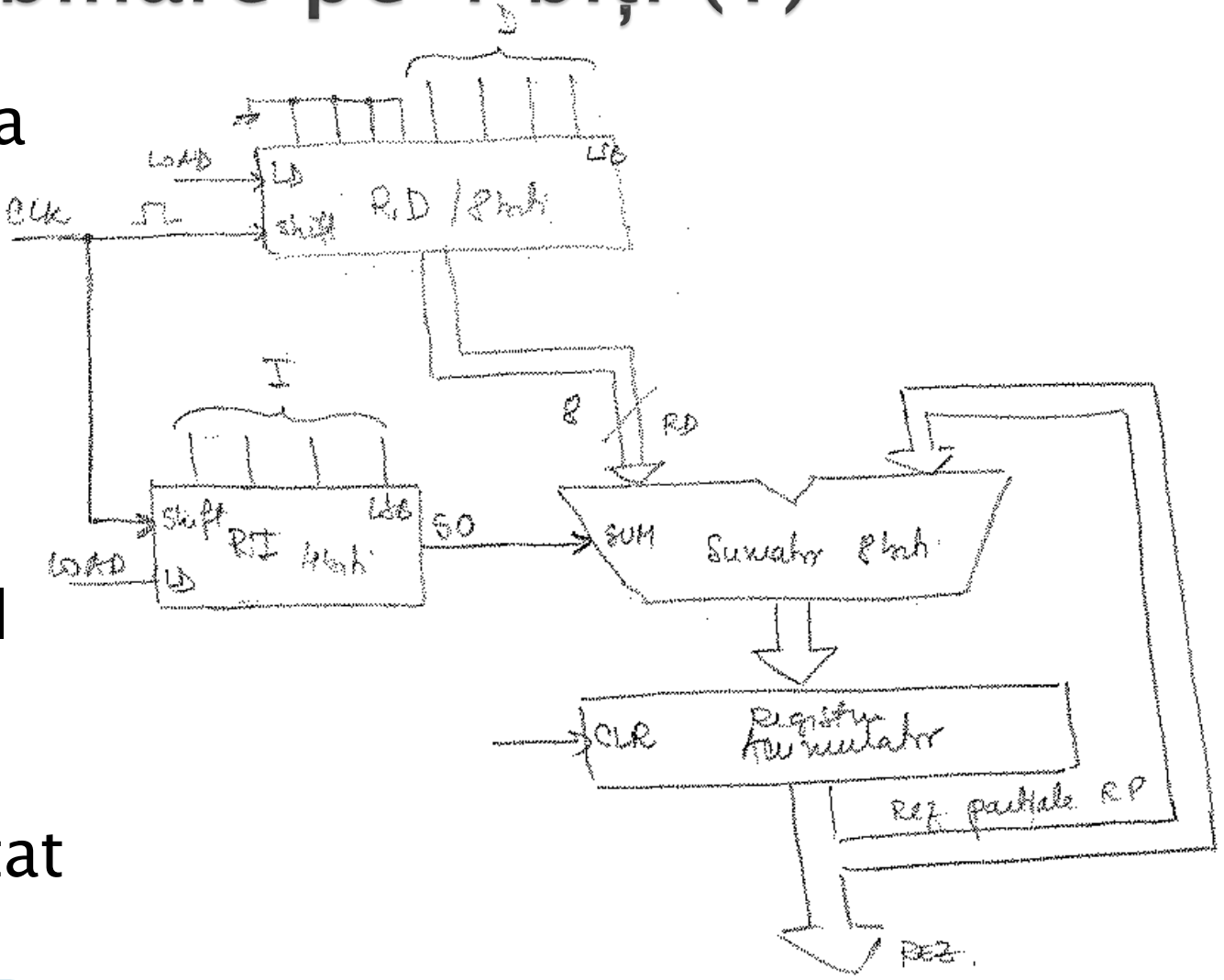
- În mod normal nu avem voie să legăm ieșirile de la 2 circuite diferite, dar acest tip de registre permit acest lucru (fac un ȘI-cablat)
- Folosit pentru a permite accesul la magistrală (transmiterea datelor de la registre la magistrală) → decid cine transmite și cine recepționează informația pe magistrală
- La activare, conținutul registrului trece pe magistrală. Dacă niciun registru nu e activ, magistrala este în TS.



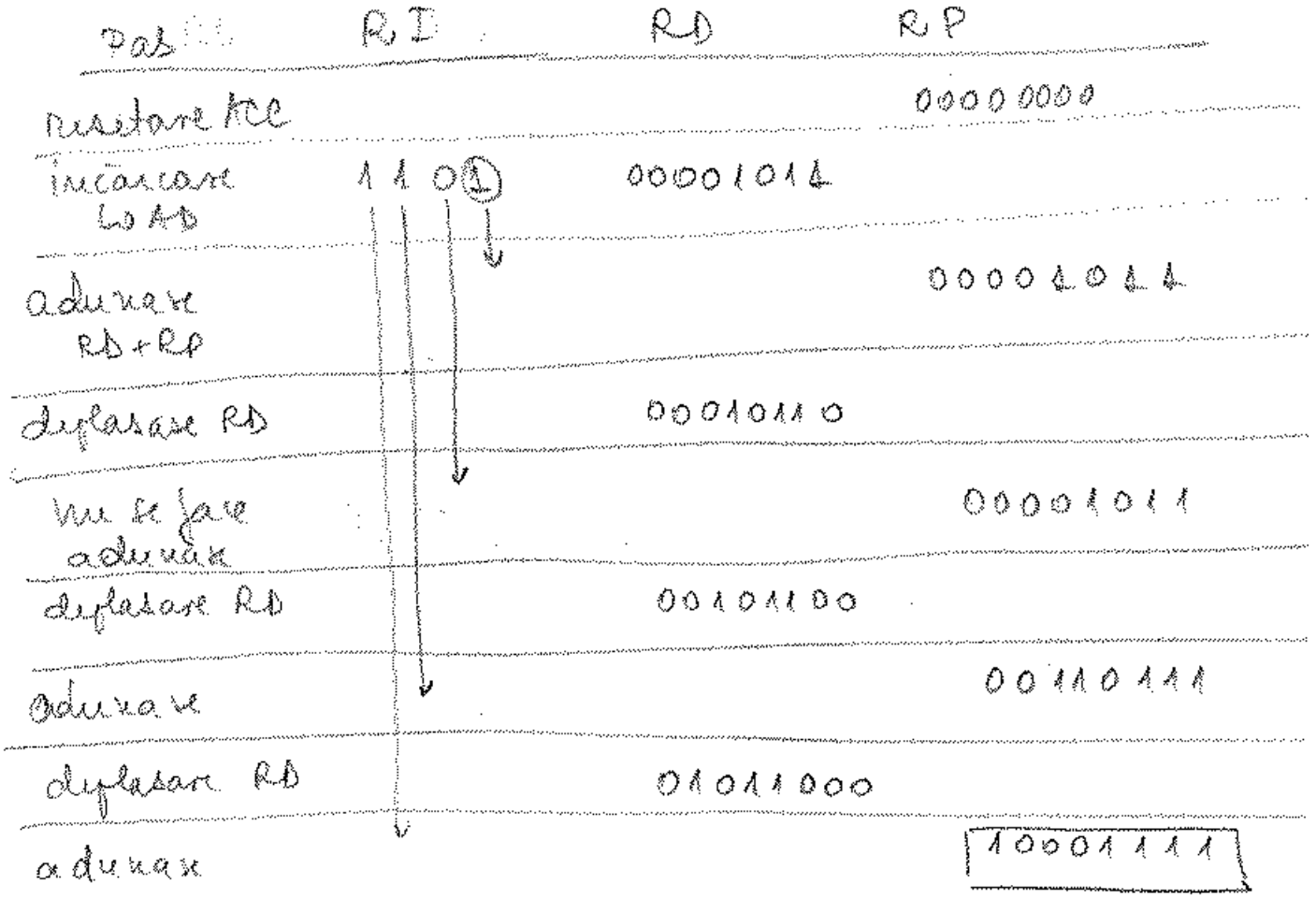


Aplicație registre: înmulțirea a 2 numere binare pe 4 biți (1)

- Rezultatul va fi pe 8 biți
- Constă în adunări și deplasări repetate
- Deînmulțitul e pe 8 biți pentru că trebuie shiftat

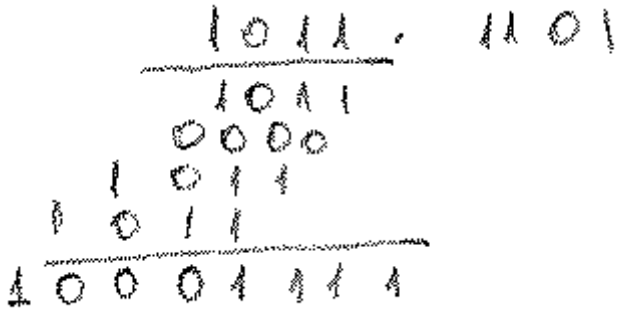


Aplicație registre: înmulțirea a 2 numere binare pe 4 biți (2)



Aplicație registre: înmulțirea a 2 numere binare pe 4 biți (3)

- Verificare



ÎNTREBĂRI?